

MÉTODOS DE CONFIABILIDAD Y CIBERSEGURIDAD

Prueba técnica — Análisis de código C (CERT C / MISRA C)

Realizado por: Luis Enrique Padilla Belmonte

Fecha:
04/05/2026

CONTENIDO

Contenido.....	2
1 Enfoque y metodología	3
2 Hallazgos sistémicos	4
2.1 Falta de validación de parámetros en funciones públicas.....	4
2.2 Comunicación y evaluación deficiente de errores.....	4
2.3 Inicialización inconsistente de variables.	5
2.4 Uso de funciones inseguras de C estándar.....	5
2.5 Ausencia de protección de concurrencia.	5
3 Hallazgos puntuales (análisis manual y estático).....	6
4 Correcciones aplicadas y desviaciones	8
4.1 Correcciones implementadas.....	8
4.1.1 Commit 1 — main.c: Alineación con buenas prácticas de desarrollo.	8
4.1.2 Commit 2 — crc.c: Resolución de vulnerabilidad de lectura fuera de límites y validación NULL.....	8
4.1.3 Commit 3 — buffer.c: validaciones de NULL e inicialización.	8
4.1.4 Commit 4 — buffer.c: manejo de errores y memoria.	9
4.1.5 Commit 5 — main.c: comprobaciones de retorno.	9
4.1.6 Commit 6 — buffer.c: protección de concurrencia.	9
4.1.7 Commit 7 — config.c: validaciones, errores y refactor de strings.	10
4.1.8 Commit 8 — config.c y main.c: serialización determinista e integridad CRC.....	10
4.1.9 Commit 9 — Correcciones de análisis estático.	10
4.2 Desviaciones justificadas.....	11
5 Análisis de integridad mediante CRC	12
5.1 Planteamiento	12
5.2 Defectos en la implementación original	12
5.3 Vista determinista de la configuración	12
5.4 Violaciones que invalidan el CRC	13
5.5 Implementación del flujo de integridad.....	13
<i>Anexo A:</i>	13

1 ENFOQUE Y METODOLOGÍA

La forma en que afronté este desafío comenzó con definir fases de acción claras, donde la primera de todas como prerequisite era familiarizarse con conceptos clave y patrones peligrosos en C.

El objetivo era llegar al código con un mapa mental de patrones peligrosos ya construido, de modo que el análisis posterior fuera dirigido. Esta fase produjo una lista de referencia de patrones de riesgo clasificados por categoría (seguridad, robustez, mantenibilidad) que actuó como criterio de detección durante todo el análisis (ver Anexo A).

El análisis manual del código lo abordé en dos pasadas. La primera fue deliberadamente lenta: Empecé leyendo cada módulo en orden de complejidad creciente (crc.c → buffer.c → config.c → main.c), prestando atención al flujo de control completo antes de entrar en defectos concretos. Esta secuencia permitió entender la arquitectura del sistema antes de auditar sus partes, comprendiendo así el propósito general del programa. La segunda pasada se centró en la identificación de hallazgos tomando como base comparativa la lista realizada previamente de "Patrones peligrosos". El análisis manual produjo inicialmente bastantes candidatos, en parte porque ante la incertidumbre se anotó todo patrón sospechoso para estudio posterior. La reducción a los ~15 hallazgos implementados y los reportados en este informe se hizo aplicando dos filtros: impacto demostrable en seguridad, robustez o mantenibilidad; y visión sistémica, es decir, si el defecto atravesaba varios módulos o comprometía el comportamiento del sistema como conjunto. Los hallazgos que no superaban ninguno de los dos filtros se documentan en la tabla de hallazgos puntuales, pero no se corrigieron en código.

El análisis se complementó con una posterior fase de validación donde realicé análisis estático para contrastar y confirmar hallazgos ya identificados manualmente o detectar los que se hubieran podido escapar. El análisis estático consistió en preparar el entorno con flags estrictos (gcc -Wall -Wextra -Wpedantic -std=c11 -g) para antes de ejecutar cualquier análisis verificar que el código compilaba correctamente. Esto garantiza que el análisis estático parte de una base compilable y que los warnings del compilador son una primera capa de detección. Posteriormente se ejecutó cppcheck con la siguiente configuración `cppcheck --enable=all --inconclusive --std=c11 -lsrc src/` donde:

- `enable=all` — activa todas las categorías: errores, warnings, estilo, rendimiento y portabilidad
- `inconclusive` — incluye hallazgos que cppcheck no puede confirmar al 100% pero que son sospechosos
- `std=c11` — alinea el análisis con el estándar C usado en el proyecto
- `lsrc` — indica dónde están los headers del proyecto

El análisis no se ejecutó una sola vez — se repitió tras cada bloque de correcciones para verificar que los hallazgos desaparecían y que las correcciones no introducían nuevos defectos. Este ciclo fue: Analizar → Identificar → Corregir → Recompilar → Volver a analizar.

Las correcciones se implementaron en commits atómicos de 3-4 cambios como máximo, recompilando entre cada uno. La razón es la misma que justifica los tests de regresión: introducir varios cambios simultáneos convierte cualquier nuevo fallo en un problema de diagnóstico, no de corrección. El orden seguido fue de menor a mayor complejidad y riesgo de efecto colateral: primero correcciones triviales y aisladas, después validaciones de NULL, después manejo de errores, y finalmente el refactor de concurrencia y la implementación de la integridad mediante CRC.

2 HALLAZGOS SISTÉMICOS

El análisis puntual hallazgo por hallazgo es necesario, pero insuficiente. Cuando un mismo tipo de defecto aparece de forma recurrente en distintos módulos, el problema no es el bug concreto sino una decisión de diseño o una práctica ausente que lo genera sistemáticamente.

Clasificar de esta manera estos hallazgos permite priorizar correcciones con mayor impacto y entender por qué el código, en su estado original, no se alinea con los principios fundamentales de CERT C ni MISRA C:2012. El criterio para clasificar un hallazgo como sistémico fue: que el patrón se repitiera en varios archivos o funciones, así como que su corrección afectara a múltiples funciones simultáneamente, que el riesgo viniera de la arquitectura del módulo y no de una línea concreta y, por último, que el problema no tuviera un origen puntual, sino que nunca se hubiera definido cómo abordarlo desde el diseño.

2.1 Falta de validación de parámetros en funciones públicas.

Ninguna función pública del proyecto valida los punteros que recibe antes de usarlos. El patrón afecta a `crc.c`, `buffer.c` y `config.c` donde desreferencian sus parámetros sin comprobar si son NULL. La única excepción es `config_crc_view_serialize`, que aplica el patrón correcto y sirve de referencia. El hallazgo es sistémico porque no se trata de descuidos puntuales sino de una decisión de diseño ausente: ningún módulo define cómo deben validarse las entradas antes de procesarlas, trasladando implícitamente esa responsabilidad al llamador. En un sistema con varios módulos interconectados, esto es frágil: cualquier llamador que no conozca esa precondition implícita provocará comportamiento indefinido sin posibilidad de detección temprana.

Normas aplicables:

CERT C EXP34-C: desreferencia de punteros sin validación previa.

CERT C API00-C: funciones públicas deben validar sus parámetros.

MISRA C:2012 Regla 17.5: parámetros punteros no deben ser NULL.

Recomendación: Añadir validación de NULL al inicio de cada función pública como primera línea de defensa, siguiendo el patrón ya presente en `config_crc_view_serialize`. Se corrigió en todos los módulos excepto la validación de `len == 0` como caso límite, documentado como hallazgo menor (ver 3. Hallazgos puntuales).

2.2 Comunicación y evaluación deficiente de errores.

Este patrón tiene dos caras que se refuerzan mutuamente. Por un lado, funciones en `buffer.c` y `config.c` que pueden fallar devuelven siempre 0 sin comprobación real de si hay error presente, convirtiendo cualquier fallo interno en un éxito silencioso. Por otro, `main.c` no comprueba ningún código de retorno en el código original.

Es sistémico porque ninguno de los dos lados es un descuido puntual: refleja una ausencia de manejo de errores a nivel de sistema, es decir, en ningún momento se definió cómo deben comunicarse y gestionarse los errores entre módulos. El resultado es que el sistema puede continuar ejecutándose sobre un estado inválido sin ninguna señal de advertencia. En sistemas críticos el peligro no es solo que falle, sino que falle silenciosamente mientras sigue operando, tomando decisiones sobre datos corruptos.

Normas aplicables:

MISRA C:2012 Regla 17.7: valores de retorno no utilizados (funciones propias).

CERT C ERR33-C: errores de funciones de librería estándar no detectados (funciones en `main`).

Recomendación: Las funciones que pueden fallar deben comunicarlo mediante códigos de retorno significativos, y los llamadores deben evaluarlos antes de continuar. Se corrigió `buffer.c`, `config.c`, y `main.c`. Quedan sin cubrir el retorno de `rb_push` dentro de `producer` (pérdida silenciosa de datos) y la distinción granular de tipos de error (ver 5. Correcciones aplicadas y desviaciones).

2.3 Inicialización inconsistente de variables.

El sistema presenta múltiples casos de variables y campos de struct que se usan o serializan sin haber sido inicializados completamente afectando principalmente a los archivos `buffer.c`, `config.c` y `main.c`.

Es sistémico porque el patrón aparece en tres módulos distintos y en todos los casos la causa es la misma: ausencia de inicialización defensiva como práctica de diseño. La consecuencia más grave es que memoria sin inicializar entra en cálculos sensibles, siendo el caso más grave la invalidación de la propiedad determinista del CRC (ver 5. Análisis de integridad mediante CRC).

Normas aplicables:

MISRA C:2012 Regla 9.1: El valor de un objeto con duración de almacenamiento automática no se debe leer antes de que se haya asignado.

CERT C EXP33-C: No leer memoria no inicializada.

Recomendación: Inicializar structs completos con `= {0}` en declaración, variables en funciones y `memset` al inicio de la función receptora. Se corrigió en todos los casos identificados.

2.4 Uso de funciones inseguras de C estándar.

El sistema hace uso extensivo de funciones de la librería estándar que no realizan comprobación de límites: `sprintf`, `strcpy`, `strcat`, `atoi` y el antipatrón `while (!feof())`, concentrados principalmente en `config.c`. Todas comparten el mismo problema de raíz: convierten cualquier entrada externa no controlada en un vector de ataque directo.

Es sistémico porque no es un uso puntual sino un patrón consistente a lo largo del módulo de parsing, donde precisamente confluyen los datos externos menos confiables del sistema.

Normas aplicables

CERT C STR31-C: Garantizar que el espacio de almacenamiento de las cadenas tenga capacidad suficiente para los datos de caracteres y el carácter de fin de cadena.

CERT C STR07-C: Utiliza las interfaces de comprobación de límites para la manipulación de cadenas.

MISRA C:2012 Regla 21.6: No se deben utilizar las funciones de entrada/salida de la biblioteca estándar (aplica a `sprintf` en sistemas críticos).

Recomendación: Sustituir por equivalentes seguros con comprobación de límites. Se corrigió la mayoría en `config.c`. Quedan pendientes `atoi` y `sprintf` en `consumer`, documentados como desviaciones justificadas (ver 5. Correcciones aplicadas y desviaciones).

2.5 Ausencia de protección de concurrencia.

El módulo `buffer.c` está diseñado para uso multihilo, como evidencia la declaración de `head` y `tail` como `volatile`. Sin embargo, esa declaración es condición necesaria pero no suficiente: `volatile` evita optimizaciones del compilador, pero no garantiza atomicidad ni exclusión mutua. Múltiples funciones acceden y modifican variables compartidas sin ningún mecanismo de sincronización además en `main.c`, la variable `running`, compartida entre productor y consumidor, carecía incluso de `volatile` en el código original. Es sistémico porque no es un olvido puntual: se es consciente del acceso concurrente pero la protección quedó incompleta a nivel de diseño del módulo. La consecuencia más grave no es un crash inmediato sino corrupción silenciosa de datos: el productor puede sobrescribir datos no consumidos, el contador puede devolver un valor que no corresponde a ningún estado real del buffer, y el sistema puede tomar decisiones sobre datos corruptos sin ninguna señal de advertencia.

Normas aplicables

CERT C CON43-C: No permitir conflictos de datos en el código multihilo.

CERT C CON40-C: No se debe hacer referencia a una variable atómica dos veces en una expresión.

Recomendación: Añadir `mutex` al struct y proteger las secciones críticas. Se corrigió en todas las funciones afectadas y `running` se corrigió con `volatile` (ver 5. Correcciones aplicadas y desviaciones).

3 HALLAZGOS PUNTUALES (ANÁLISIS MANUAL Y ESTÁTICO)

La siguiente tabla resume los defectos identificados y corregidos durante la “auditoría” de código. Los hallazgos se han priorizado según su impacto en la seguridad (S), robustez (R) y mantenibilidad (M) del sistema.

ID	Archivo:línea	Defecto e Impacto	Tipo	Sev.	Norma	Corrección aplicada
C1	crc.c:26	Off-by-one: Lectura fuera de límites en el bucle de cálculo. Riesgo de corrupción de datos.	R	Cri.	CERT ARR30-C MISRA 18.1	Sustitución de <= por < en la condición de parada.
C2	crc.c:19	Input NULL: Función de CRC no valida el puntero de entrada. Riesgo de crash.	R	Alt.	CERT EXP34-C	Añadida guarda de seguridad al inicio de la función.
B1	buffer.c:7	Fallo de reserva: malloc sin verificar. Riesgo de Null pointer dereference y crash sistémico.	R	Cri.	CERT MEM32-C	Implementada guarda de seguridad if (rb->data == NULL).
B2	buffer.c:10	Estado inválido: Miembro tail no inicializado. El buffer inicia con índices basura.	R	Alt.	CERT EXP33-C	Inicialización explícita a cero durante el setup inicial.
B4	buffer.c:16	Puntero colgado: No se anula el puntero tras free. Riesgo de use-after-free o double-free.	S	Cri.	CERT MEM30-C	Asignación rb->data = NULL inmediatamente tras el free.
B6	buffer.c (varias)	Mutex en buffer.c: Race condition. Variables compartidas sin mutex.	S	Cri.	CERT CON43-C	Añadido pthread_mutex_t mutex al struct RingBuffer. Protección con lock/unlock en rb_init, rb_push, rb_pop, rb_count.
B7	buffer.c:44	Error de tipo: Cast explícito sin verificación de rango. Riesgo de desbordamiento de entero.	R	Med.	MISRA 10.3	Verificar el rango del cast y uso de tipos coherentes (size_t).
F1	config.c:7	Acceso inseguro: fopen sin validar. El sistema falla al intentar leer un archivo inexistente.	R	Cri.	CERT FIO42-C	Validación de puntero FILE * antes de iniciar la lectura.
F2	config.c:11	Flujo deficiente: Antipatrón while(!feof). Riesgo de procesar el último registro dos veces.	M	Med.	CERT FIO34-C	Reestructuración del flujo basada en el retorno de fgets.
F3	config.c:19	Buffer overflow: sprintf sin control de tamaño. Corrupción de memoria, crash o potencial exploit.	S	Cri.	CERT STR07-C MISRA 21.6	Sustitución por snprintf con límite estricto de búfer.
F4	config.c:24	Entorno no validado: Uso de getenv sin verificar retorno. Riesgo de excepción.	R	Alt.	CERT ENV34-C	Verificación de nulo antes de operar con la cadena obtenida.
F10	config.c:48-50	CRC no determinista: memcpy copia bytes basura tras el \0 de log_path. Anula el memset previo.	S	Cri.	CERT API00-C CERT EXP33-C MISRA 9.1	Reemplazado memcpy ciego por strncpy con strlen para copiar solo bytes válidos. Mantenido memset previo para garantizar ceros tras el \0.
M1	main.c:10	Race Condition: Variable running sin calificador de acceso. Flujo de hilos impredecible.	R	Cri.	CERT CON32-C	Adición del calificador volatile para asegurar lectura de RAM.
M2	main.c (varias)	Comprobaciones de retorno en main: comprobar en las funciones el retorno que generan y evaluarlo/tratarlo como error o éxito.	R	Cri.	CERT ERR33-C MISRA 17.7	Añadir comprobaciones de retorno de pthread, load_config,
M3	main.c:62-63	Doble liberación: Llamada redundante a rb_free. Riesgo de corrupción del heap.	S	Cri.	CERT MEM31-C CERT MEM01-C	Eliminada la segunda llamada a rb_free(&rb). La primera ya libera y deja rb->data = NULL, evitando doble free.
M4	main.c:39-41	Salida anómala: El programa continúa tras error en argumentos. Flujo inconsistente.	M	Baj.	CERT ERR04-C MISRA 17.7	Inclusión de exit(1) tras mostrar el mensaje de uso.

MÉTODOS DE CONFIABILIDAD Y CIBERSEGURIDAD

LUIS ENRIQUE PADILLA BELMONTE

M5	main.c:43	AppConfig sin inicializar: Variable usada antes de inicializar.	S	Alt.	MISRA 9.1.	AppConfig cfg = {0}
M6	main.c:53-55	CRC no implementado: Omisión de CRC sobre "hello". Brecha de integridad funcional.	S	Cri.	MISRA Regla 9.1 CERT C EXP33-C	Sustituido el CRC sobre cadena de prueba 'hello' por el CRC real sobre la vista determinista de AppConfig (config_crc_view_serialize + crc32_compute).

Tabla 1: Hallazgos puntuales.

ID	Hallazgo sistémico	Archivos afectados	Sev	Norma	Estado
S1	Falta de validación de parámetros en funciones públicas	crc.c, buffer.c, config.c	Crit.	CERT EXP34-C, API00-C, MISRA 17.5	Corregido (contiene desviaciones documentadas)
S2	Comunicación y evaluación deficiente de errores	buffer.c, config.c, main.c	Crit.	MISRA 17.7, CERT ERR33-C	Corregido (contiene desviaciones documentadas)
S3	Inicialización inconsistente de variables	buffer.c, config.c, main.c	Alt.	MISRA 9.1, CERT EXP33-C	Corregido totalmente
S4	Uso de funciones inseguras de C estándar	config.c	Crit.	CERT STR31-C, STR07-C, MISRA 21.6	Corregido (contiene desviaciones documentadas)
S5	Ausencia de protección de concurrencia	buffer.c, main.c	Crit.	CERT CON43-C, CON40-C	Corregido totalmente

Tabla 2: Hallazgos sistémicos.

Los hallazgos recogidos en la tabla anterior se identificaron mediante análisis manual con criterio de impacto demostrable en seguridad, robustez o mantenibilidad, y fueron posteriormente validados con herramientas de análisis estático.

Este análisis estático confirmó el 90% de los hallazgos manuales significativos y aportó dos hallazgos adicionales de mantenibilidad que posteriormente se corregirían. La última ejecución de cppcheck produjo salida limpia — cero warnings, cero errores, cero hallazgos de estilo sobre el código fuente del proyecto. El único aviso restante es el warning generado por las flags de GCC de truncación de snprintf en config.c, documentado como desviación justificada ya que el comportamiento es controlado y seguro.

La justificación técnica de los hallazgos se sustenta en la violación de la norma referenciada y en el impacto descrito en la propia tabla. Las correcciones aplicadas se detallan en la sección 4. Los hallazgos no implementados se documentan como propuestas o desviaciones justificadas en la misma sección (ver 4. Correcciones aplicadas y desviaciones).

4 CORRECCIONES APLICADAS Y DESVIACIONES

Las correcciones se implementaron en commits atómicos ordenados de menor a mayor complejidad, recompilando tras cada uno para verificar que no se introducían nuevos defectos.

Las correcciones se agruparon por archivo y fase de análisis, siguiendo un criterio de menor a mayor complejidad y riesgo de efecto colateral. Primero se aplicaron cambios triviales y aislados sin impacto en el flujo lógico, después validaciones de NULL, después manejo de errores, y finalmente el defecto de concurrencia y la implementación de integridad mediante CRC.

Esta secuencia tiene una justificación técnica concreta: introducir varios cambios simultáneos convierte cualquier nuevo fallo en un problema de diagnóstico en lugar de corrección. Aplicar cambios atómicos y recompilar entre cada commit permite atribuir cualquier warning o error nuevo al cambio exacto que lo introdujo, lo que reduce el riesgo de regresión y facilita la revisión del historial de cambios.

4.1 Correcciones implementadas

4.1.1 Commit 1 — *main.c*: Alineación con buenas prácticas de desarrollo.

Implementación	Hallazgo resuelto
Añadir static a producer y consumer	BP — Encapsulación — funciones internas expuestas
Declarar data como const	BP — Inmutabilidad — dato de solo lectura sin protección
Inicializar AppConfig cfg = {0}	M5 — AppConfig sin inicializar
Eliminar segundo rb_free	M3 — Doble liberación
Añadir return 1 tras printf("Usage:")	M4 — Salida anómala sin retorno
Añadir volatile a running	M1 — Race condition en variable compartida

Tabla 3: Commit 1 — Implementaciones..

Ninguno de estos cambios altera el flujo lógico del programa: son correcciones de visibilidad, inmutabilidad e inicialización con efecto colateral nulo. Aplicarlos primero permite establecer una base limpia antes de abordar cambios más profundos.

4.1.2 Commit 2 — *crc.c*: Resolución de vulnerabilidad de lectura fuera de límites y validación NULL.

Implementación	Hallazgo resuelto
Sustituir <= por < en condición del bucle	C1 — Off-by-one, lectura fuera de límites
Validar data != NULL al inicio de crc32_compute	C2 — Input NULL sin validar

Tabla 4: Commit 2 — Implementaciones.

La corrección aplicada elimina los hallazgos C1 y C2 (ver Tabla 1), asegurando que el cálculo del CRC opere únicamente sobre memoria válida. La modificación del límite del bucle evita una lectura fuera de rango, y la validación del puntero de entrada previene fallos inmediatos por desreferenciación nula.

4.1.3 Commit 3 — *buffer.c*: validaciones de NULL e inicialización.

Implementación	Hallazgo resuelto
Validar NULL en rb_init, rb_free, rb_push, rb_pop, rb_count	S1 — Falta de validación de parámetros
Descomentar e inicializar rb->tail = 0U	B2 — Estado inválido, tail no inicializado

Tabla 5: Commit 3 — Implementaciones.

En este paso se aborda el hallazgo sistémico S1 junto con el hallazgo puntual B2 (ver Tablas 1 y 2). La introducción de validaciones defensivas en las funciones públicas del módulo evita fallos por uso incorrecto desde otros componentes, mientras que la inicialización explícita de tail garantiza un estado inicial coherente del buffer circular.

4.1.4 Commit 4 — *buffer.c*: manejo de errores y memoria.

Implementación	Hallazgo resuelto
Comprobar resultado de malloc en rb_init	B1 — Fallo de reserva sin verificar
Asignar rb->data = NULL tras free en rb_free	B4 — Puntero colgado tras free

Tabla 6: Commit 4 — Implementaciones.

La implementación refuerza la gestión de memoria del módulo corrigiendo los hallazgos B1 y B4 (ver Tabla 1). La comprobación del resultado de malloc evita accesos a punteros nulos, y la anulación explícita del puntero tras free elimina escenarios de use-after-free o liberaciones duplicadas.

4.1.5 Commit 5 — *main.c*: comprobaciones de retorno.

Implementación	Hallazgo resuelto
Comprobar retorno de load_config	M2 — Retorno ignorado
Comprobar retorno de rb_init	M2 — Retorno ignorado
Comprobar retorno de pthread_create (ambos hilos)	M2 — Retorno ignorado
Gestionar fallo del segundo pthread_create con pthread_join del primero	M2 — Hilo huérfano
Definir BUFFER_SIZE como constante con nombre	BP — Número mágico (MISRA 2.4)

Tabla 7: Commit 5 — Implementaciones.

Con este conjunto de cambios se corrige el hallazgo sistémico S2 y el hallazgo puntual M2 (ver Tablas 1 y 2), estableciendo un manejo coherente de errores a nivel de sistema. La evaluación sistemática de códigos de retorno impide que el programa continúe ejecutándose sobre estados inválidos, y el tratamiento específico del fallo en la creación de hilos garantiza una terminación limpia.

4.1.6 Commit 6 — *buffer.c*: protección de concurrencia.

Implementación	Hallazgo resuelto
Añadir pthread_mutex_t mutex al struct RingBuffer	B6 — Race condition
Inicializar mutex en rb_init, destruirlo en rb_free	B6 — Race condition
Proteger secciones críticas en rb_push, rb_pop, rb_count	B6 — Race condition
Eliminar const de rb_count para permitir uso de mutex	Trade-off documentado como desviación

Tabla 8: Commit 6 — Implementaciones.

La integración de un mutex en el RingBuffer resuelve el hallazgo S5 / B6 (ver Tablas 1 y 2), eliminando condiciones de carrera en el acceso concurrente. La protección se aplica tanto a lecturas como escrituras de índices compartidos. La retirada del calificador const en rb_count se documenta como una desviación justificada, priorizando la seguridad frente a concurrencia sobre la inmutabilidad semántica.

4.1.7 Commit 7 — *config.c: validaciones, errores y refactor de strings.*

Implementación	Hallazgo resuelto
Validar path y cfg al inicio de load_config	S1 — Falta de validación de parámetros
Inicializar cfg con memset	S3 — Inicialización inconsistente
Comprobar retorno de fopen	F1 — Acceso inseguro sin validar
Mover fclose antes de cada return de error	F1 — Recurso no liberado
Sustituir while (!feof) por while (fgets() != NULL)	F2 — Antipatrón de lectura
Sustituir sprintf por snprintf	F3 — Buffer overflow
Validar retorno de getenv antes de usar	F4 — Entorno no validado
Sustituir strcpy por strncpy con terminador forzado	S4 — Función insegura sin límite
Sustituir strcat por strncat con espacio restante calculado	S4 — Función insegura sin límite

Tabla 9: Commit 7 — Implementaciones.

Las modificaciones introducidas corrigen los hallazgos S1, S3, F1, F2, F3 y F4 (ver Tablas 1 y 2), reforzando el módulo encargado del procesamiento de entrada externa. La combinación de validaciones, inicialización defensiva y sustitución de funciones inseguras reduce significativamente la superficie de fallo y de ataque.

4.1.8 Commit 8 — *config.c y main.c: serialización determinista e integridad CRC.*

Implementación	Hallazgo resuelto
Inicializar ConfigCrcView v con memset(&v, 0, sizeof v)	F10 — CRC no determinista
Sustituir memcpy ciego por strncpy con strlen	F10 — CRC no determinista
Sustituir flag implícito crc_table[1] == 0 por table_initialized explícito en crc32_compute	Determinismo de tabla CRC
Implementar CRC real sobre AppConfig en main	M6 — CRC no implementado
Corregir comparación != 0 por < 0 en llamada a config_crc_view_serialize	Detección incorrecta de error (detectado al compilar)

Tabla 10: Tabla 8: Commit 8 — Implementaciones.

La corrección aplicada garantiza que el CRC dependa exclusivamente de los datos reales de la configuración, resolviendo los hallazgos F10 y M6 (ver Tabla 1). La serialización determinista elimina dependencias de padding y memoria residual, y la implementación del CRC real sustituye la comprobación ficticia original, aportando integridad funcional al sistema.

4.1.9 Commit 9 — *Correcciones de análisis estático.*

Implementación	Hallazgo resuelto
Añadir tmp[sizeof(tmp)-1] = '\0' tras strncpy	terminateStrncpy — buffer sin terminador garantizado
Simplificar ternario (argc > 1) ? argv[1] : ... a argv[1]	knownConditionTrueFalse — condición siempre verdadera
Cambiar char* resultado a const char*	constVariablePointer — getenv devuelve const

Tabla 11: Commit 9 — Implementaciones.

Finalmente, se incorporan correcciones adicionales de mantenibilidad y seguridad detectadas en la última ejecución de análisis estático. La garantía explícita del terminador nulo tras `strncpy` evita lecturas fuera de límites, y la eliminación de código redundante mejora la claridad del código sin impacto funcional.

4.2 Desviaciones justificadas

Durante el análisis se identificaron ciertos incumplimientos de reglas MISRA/CERT que, tras evaluación del contexto y del impacto, se decidieron mantener de forma consciente y documentada.

D1 — Eliminación de `const` en `rb_count` en el archivo `buffer.c` línea 44

Se elimina el calificador `const` del parámetro `RingBuffer *` para permitir la protección mediante `mutex`. La seguridad frente a condiciones de carrera tiene prioridad sobre la inmutabilidad semántica. Regla 17.1 de MISRA C:2012 (Advisory).

D2 — Posible truncación en `snprintf`

El uso de `snprintf` garantiza seguridad de memoria. La posible truncación afecta solo a funcionalidad secundaria y no compromete la robustez del sistema. Aumentar el tamaño del buffer implicaría cambios estructurales fuera del alcance del ejercicio.

D3 — Uso de `atoi` en parsing numérico

Aunque `atoi` no permite detectar errores de conversión, se mantiene por simplicidad y control del formato de entrada. Se documenta la recomendación de usar `strtol` en una versión futura.

D4 — Múltiples puntos de retorno

Se mantiene el patrón de retorno temprano por claridad y mantenibilidad. La regla MISRA 15.5 es Advisory y la estructura actual es segura y fácilmente auditable.

Recomendaciones y mejoras no implementadas:

Durante el análisis se identificaron algunos aspectos mejorables que no constituyen desviaciones de MISRA/CERT en el contexto del ejercicio, pero que podrían considerarse en una evolución futura del sistema:

- Gestión de errores en `crc32_compute`: la función devuelve 0 tanto en caso de error como para un CRC válido. Una interfaz más robusta separaría el código de error del valor calculado, por ejemplo, mediante un parámetro de salida o un tipo de retorno estructurado.
- Portabilidad de tipos en `AppConfig`: el uso de `int` depende del tamaño de palabra de la arquitectura. Para maximizar portabilidad entre plataformas, se recomienda emplear tipos de tamaño fijo (`int32_t`) en estructuras que puedan ser serializadas o persistidas.
- Dependencia de variables de entorno (`getenv`): el uso de variables de entorno es específico de entornos tipo Unix. En sistemas industriales heterogéneos, convendría abstraer esta funcionalidad o limitarla mediante una lista blanca controlada.

Estas recomendaciones no se implementaron al no ser críticas para la seguridad o robustez inmediata del sistema, pero se documentan como mejoras de calidad y mantenibilidad.

5 ANÁLISIS DE INTEGRIDAD MEDIANTE CRC

5.1 Planteamiento

El objetivo de esta sección es evaluar e implementar un mecanismo de integridad para la configuración del sistema (AppConfig) mediante un CRC, tal como solicita el enunciado.

El CRC se utiliza para detectar corrupciones accidentales de datos (memoria degradada, errores de escritura, lecturas inconsistentes), no como mecanismo criptográfico frente a ataques deliberados.

En sistemas embebidos e industriales, verificar la integridad de la configuración es crítico: una configuración corrupta puede provocar comportamientos no deterministas o decisiones incorrectas del sistema incluso aunque el software “siga funcionando”.

5.2 Defectos en la implementación original

La implementación original del proyecto no protegía realmente la configuración:

El CRC se calculaba sobre una cadena de prueba ("hello"), sin relación con AppConfig.

La función diseñada para serializar la configuración (config_crc_view_serialize) no se utilizaba.

El cálculo del CRC no estaba ligado a una representación estable de los datos reales.

Además, el análisis manual reveló un problema más grave: aunque se usara AppConfig directamente, el CRC no sería determinista, debido a:

- bytes de memoria no inicializada
- uso de copias ciegas (memcpy)
- posible padding en estructuras

Esto invalida el CRC como mecanismo de integridad.

5.3 Vista determinista de la configuración

Para que un CRC sea válido, debe calcularse sobre una vista determinista, es decir, una representación binaria que dependa únicamente de los datos lógicos reales y no del estado accidental de la memoria.

La solución adoptada sigue estos principios:

- Uso de una estructura intermedia (ConfigCrcView) con:
 - tipos de tamaño fijo (int32_t)
 - orden y tamaño conocidos
- Inicialización explícita de toda la estructura con memset
- Copia controlada de cadenas:
 - solo hasta el carácter '\0'
 - dejando a cero los bytes restantes

Nota sobre padding:

Aunque el padding en estructuras es una fuente habitual de no determinismo en el cálculo de CRC, en este proyecto concreto se verificó que la estructura AppConfig, debido al orden y tipo de sus campos en la arquitectura objetivo, no introduce bytes de relleno efectivos.

No obstante, el diseño no confía en esta propiedad circunstancial: el cálculo del CRC se realiza siempre sobre una vista serializada (ConfigCrcView) con tipos de tamaño fijo y memoria completamente inicializada, garantizando determinismo incluso si la estructura original cambiara o se portara a otra arquitectura.

Esta serialización convierte AppConfig en una secuencia binaria estable y reproducible, apta para el cálculo del CRC.

5.4 Violaciones que invalidan el CRC

La siguiente tabla resume qué tipos de defectos rompen la integridad del CRC y por qué representan un riesgo real:

Violación	Por qué afecta al CRC	Riesgo
Padding en estructuras	Se incluyen bytes no definidos	CRC no determinista
Variables no inicializadas	Memoria basura entra en el cálculo	Falsos positivos/negativos
Copias ciegas (memcpy)	Se copian bytes residuales	CRC dependiente del azar
Tipos dependientes de arquitectura (int)	Tamaño variable entre plataformas	CRC no portable
Lecturas concurrentes sin protección	Estado inconsistente durante serialización	CRC calculado sobre datos intermedios

Tabla 12: Defectos CRC.

Estas violaciones están directamente relacionadas con reglas de MISRA C (9.1) y CERT C (STR31-C, EXP33-C, CON40-C).

5.5 Implementación del flujo de integridad

El flujo final implementado es:

1. Cargar la configuración (load_config)
2. Serializar AppConfig a una vista determinista (config_crc_view_serialize)
3. Calcular el CRC sobre esa vista (crc32_compute)
4. Obtener una huella reproducible de la configuración

Con esta implementación:

- La misma configuración produce siempre el mismo CRC
- Cualquier modificación mínima de los datos produce un CRC distinto
- El CRC deja de depender del estado previo de la memoria

En conclusión, la implementación del CRC sobre una vista determinista permite detectar de forma fiable corrupciones accidentales de la configuración, eliminando dependencias de memoria no inicializada, padding y concurrencia. Este mecanismo mejora la robustez del sistema, aunque no sustituye a medidas criptográficas en escenarios con requisitos de autenticación o protección frente a ataques deliberados.

Anexo A: Salidas del compilador — Pruebas y evidencias.

```
PS C:\Users\SaoYLuis\Documents\pruebaTecnica\FZM-prueba-static-analysis-v1> .\bin\app.exe .\tests\example.cfg
CRC=78366692
PS C:\Users\SaoYLuis\Documents\pruebaTecnica\FZM-prueba-static-analysis-v1> .\bin\app.exe .\tests\example.cfg
CRC=78366692
PS C:\Users\SaoYLuis\Documents\pruebaTecnica\FZM-prueba-static-analysis-v1> .\bin\app.exe .\tests\example.cfg
CRC=78366692
PS C:\Users\SaoYLuis\Documents\pruebaTecnica\FZM-prueba-static-analysis-v1> .\bin\app.exe .\tests\example.cfg
CRC=78366692
PS C:\Users\SaoYLuis\Documents\pruebaTecnica\FZM-prueba-static-analysis-v1> .\bin\app.exe .\tests\example.cfg
CRC=d63d3506
PS C:\Users\SaoYLuis\Documents\pruebaTecnica\FZM-prueba-static-analysis-v1> .\bin\app.exe .\tests\example.cfg
CRC=d63d3506
PS C:\Users\SaoYLuis\Documents\pruebaTecnica\FZM-prueba-static-analysis-v1> .\bin\app.exe .\tests\example.cfg
CRC=d63d3506
PS C:\Users\SaoYLuis\Documents\pruebaTecnica\FZM-prueba-static-analysis-v1> .\bin\app.exe .\tests\example.cfg
CRC=78366692
PS C:\Users\SaoYLuis\Documents\pruebaTecnica\FZM-prueba-static-analysis-v1> .\bin\app.exe .\tests\example.cfg
CRC=78366692
```